

Inmatning från tangentbordet

Förutom att endast skriva ut data till skärmen vill man förr eller senare kunna göra interaktiva program som tar input från användaren via tangentbordet. Även detta är lätt att göra i C++. För input finns en global variabel liknande `cout` som heter `cin`. Största skillnaden är att man tillsammans med `cin` använder sig av `>>`. Man kan alltså se dem som "pilar" som anger åt vilket håll dataströmmen är riktad. Denna dataström brukar kallas *standard in*, i motsats till *standard out*.

Inmatning i C++ sköts direkt in i variabler, d.v.s `<< "pekar"` på en variabel som skall tilldelas det inlästa värdet. Allmänt ser det ut så här:

```
cin >> variabel;  
cin >> variabel1 >> variabel2 >> ... >> variabelN;
```

Det är alltså möjligt att även läsa in flera värden samtidigt. Värdena läses in från vänster till höger. Ett enkelt exempel som läser in ett tecken i taget tills ett visst tecken givits är:

```
char Tecken;  
  
// upprepa slingan ända tills användaren ger in tecknet 'q'  
do {  
    // läs in ett tecken  
    cout << "Ge ett tecken: ";  
    cin >> Tecken;  
    cout << "Du gav in tecknet '" << Tecken << "'." << endl;  
}  
while ( Tecken != 'q' );
```

Om du provar programmet i verkligheten kommer du att märka att programmet inte reagerar trots att du skriver ett tecken. Inget händer fast flera tecken skrivs. Input i C++ är buffrat *radvis*, och en rad avslutas av att *enter* trycks. Efter detta skickas hela raden till programmet som sedan får en buffer som innehåller ett eller flera tecken som kan behandlas. Om du t.ex. ger in tre tecken och sedan trycker enter kommer du att få tre rader utskrift. När programmet behandlat alla tecken som skrivits skriv en prompt ut på nytt och programmet väntar sig mera input. Notera dock att även varje gång som input fanns färdigt i buffern skrivs en prompt ut, men användaren ges aldrig ett tillfälle att mata in data. Ett exempel på hur ovanstående program kan fås att läsa två tecken samtidigt:

```
char Tecken1, Tecken2;  
  
// upprepa slingan ända tills användaren ger in tecknet 'q'  
do {  
    // läs in ett tecken  
    cout << "Ge två tecken: ";  
    cin >> Tecken1 >> Tecken2;  
    cout << "Du gav in tecknen '" << Tecken1 << "' och '" << Tecken2 << "'." << endl;  
}  
while ( Tecken1 != 'q' && Tecken2 != 'q' );
```

Även här skrivs prompten ut ifall flera än två tecken givits. Programmet kommer att behandla två tecken åt gången, så om t.ex. tre tecken ges in första gången räcker det andra gången med att ge in ett enda tecken. Ges endast ett tecken första gången händer ingenting utan programmet väntar sig ett

tecken till följt av *enter*.

Förutom tecken kan man från `cin` även läsa in andra typer av data, t.ex. `int`, `string` och `float`. Ett enkelt exempel som läser in dessa datatyper ser ut så här:

```
int Tal;
float Flyttal;
string Text;

// läs ett tal
cout << "Ge ett heltal: " << flush;
cin >> Tal;

// läs ett decimaltal
cout << "Ge ett decimaltal: " << flush;
cin >> Flyttal;

// läs en sträng
cout << "Ge en sträng: " << flush;
cin >> Text;

cout << "Du gav värdena '"
    << Tal << "', '"
    << Flyttal << "', '"
    << Text << "'.'" << endl;
```

Om man provar programmet och ger in ett illegalt värde på t.ex. heltalet kommer programmet att försöka tolka talet som ett heltal, men utan att lyckas. Följden blir för det mesta ett felaktigt värde och att de övriga värdena som läses in även får illegala värden. Strängen som läses in sist accepterar vilka värden som helst, eftersom en sträng inte försöker konvertera den lästa texten på något sätt. Om du försöker ge in en sträng som består av flera än ett enda ord kommer du att märka att strängen bryts vid det första mellanslaget eller tomrummet. Mellanslag används för att avsluta inläsning av en sträng. Du kommer att märka att det kan vara knepigt att göra ett program som läser input från användaren på ett robust sätt, speciellt om många olika datatyper används och programmet bör klara av att hantera olika typer av illegala värden. Det är inte omöjligt, men det är svårt att göra en robust lösning.

Notera även att C++ tillåter att decimaltal matas in enligt den "vetenskapliga" notationen, t.ex. följande tal är godtagbara flyttal:

```
1.05
-0.0003
9.8e5
-0.0001e-1
6.02e24
```

[Förutgående](#)
Utskrift till skärmen

[Hem](#)
[Upp](#)

[Nästa](#)
Utskrift till `cerr` och `clog`